# Bangladesh University of Business and Technology

## ASSIGNMENT

**Assignment No** : 01
**Course Code** : CSE 207
**Course Title** : Database Systems


**Submitted By:**

**Name :** Luckman Rafique
**ID No :** 20255103042
**Intake :** 55
**Section :** 02
**Program :** B.Sc. in CSE

**Submitted To:**

**Name :** Md. Shakil Ahmed

*(Lecturer)*
**Dept. of** CSE
**Bangladesh University of Business & Technology**


**Date of Submission :** 11/10/2026

SET A

Answer to the question no: 1

Two-tier Architecture :-

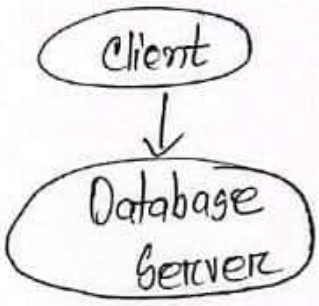
The client communicates directly with the database server.

Components –

(i) Client : Provides the user interface.

(ii) Database Server : Stores and manages data.

Communication –




Three-tier Architecture :-

The client communicates with the database through an application server

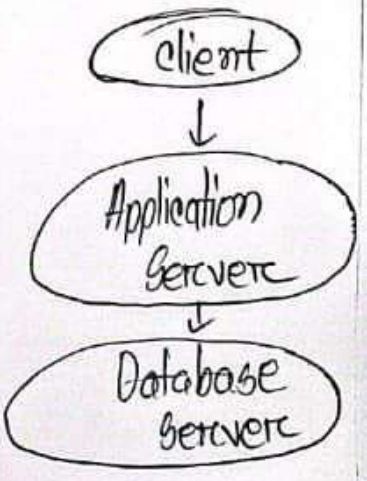
Components –

(i) Presentation tier : Web browser used by user

(ii) Application tier : Process requests and business logic.

(iii) Data tier : Stores and manages database records.

Communication –




Three-tier architecture is more appropriate for the scenario. Because it provides better security, scalability, easier maintenance and efficient management of concurrent req.

Set-A

Answer to the question no: 2

(a)
```
CREATE TABLE Course (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    category VARCHAR(50),
    start_date DATE,
    instructor_id INT,
    FOREIGN KEY (instructor_id)
        REFENGES Instructor(i_id)
);
```

(b)
```
SELECT DISTINCT I.i_name
FROM Instructor I JOIN Course C
ON I.i_id = C.instructor_id
WHERE C.category = 'Programming';
```

(c)
```
SELECT C.course_id, C.course_name,
COUNT(CS.s_id) AS total_students
FROM Course C
LEFT JOIN Course_Student CS
ON C.course_id = CS.course_id
GROUP BY C.course_id, C.course_name;
```

SET A

(d)
```
SELECT category,
    COUNT(*) AS total_courses
FROM Course
GROUP BY category
HAVING COUNT(*)>2;
```

(e)
```
SELECT I.i_name, COUNT (c.course_id) AS
total_courses FROM Instructor I JOIN Course C
ON I.i_id = c.instructor_id
Group BY I.i_id, I.i_name
HAVING COUNT (c.course_id) >= ALL (
    SELECT COUNT (*)
    FROM Course
    GROUP BY instructor_id
);
```

SET A

Answer to the question no: 3

(a) $\Pi_{course_name,\ i_name}$ (Course $\bowtie_{instructor_id = i_id}$ Instructor)

(b) $\Pi_{i_name}$ ($\sigma_{category = 'Database'}$ (Course $\bowtie_{instructor_id = i_id}$ Instructor))

(c) $\Pi_{s_name,\ course_name,\ i_name}$ (Student $\bowtie_{student.s_id = course_student.s_id}$ Course_Student $\bowtie_{Course_Student.course_id = Course.course_id}$ (Course $\bowtie_{Course.instructor_id = Instructor.i_id}$ Instructor)) ,le

(d) $R \leftarrow \gamma_{course_id;\ COUNT(s_id) \rightarrow total_students}$ (Course_Student)

Now,

$\Pi_{Course.course_id,\ course_name,\ total_students}$ (Course $\bowtie$ R)

(e) $R \leftarrow \gamma_{category;\ COUNT(course_id) \rightarrow total}$ (Course)

Now,

$\sigma_{total > 3}$ (R)

SET B

Answer to the question no: 1

Example of DDL commands -

(i) CREAT : creates a new table

```
CREAT TABLE Student (
    id int,
    name varchar(50),
    age int
);
```

(ii) ALTER : modifies the structure of an existing table

```
ALTER TABLE Students
ADD email varchar(100);
```

(iii) DROP : deletes a table and its structure

```
DROP TABLE student;
```

SET B

Examples of DML (Data Manipulation Language) -

(i) INSERT : adds new record to a table

```
INSERT INTO Student (id, name, age)
VALUES (101, 'Rahim', 20);
```

(ii) UPDATE : modifies existing records

```
UPDATE Student SET age=22 WHERE id=101;
```

(iii) DELETE : removes records from a table

```
DELETE FROM Student WHERE id=101;
```

(iv) SELECT : Retrives data from a table

```
SELECT * FROM Student;
```

Differences between DDL and DML

| DDL | DML |
|---|---|
| Defines database structure | Manipulates data in table. |
| Works mainly with database object. | Works with records of row. |
| Example: CREATE, ALTER, DROP. | Example: INSERT, UPDATE, DELETE, SELECT. |
| Used to create or modify tables. | Used to add, change, remove or retrive data. |

SET B

Answer to the question no: 2

(a)
```
CREATE TABLE Doctor (
    doc_id INT PRIMARY KEY,
    doc_name VARCHAR (100),
    specialization VARCHAR (50),
    joining_date DATE,
    hospital_id INT,
    FOREIGN KEY (hospital_id) REFERENCES Hospital (h_id)
);
```

(b)
```
SELECT H.h_name, COUNT (D.doc_id) AS total_doctors
FROM Hospital H JOIN Doctor D
ON H.h_id = D.hospital_id
GROUP BY H.h_id, H.h_name HAVING COUNT (D.doc_id) > 5;
```

(c)
```
SELECT D.doc_name, COUNT (DP.p_id) AS total_patients
FROM Doctor D LEFT JOIN Doctor_Patient DP
ON D.doc_id = DP.doc_id
GROUP BY D.doc_id, D.doc_name;
```

SET-B

(d)
```
SELECT D.doc_name, D.specialization
FROM Doctors D WHERE D.hospital_id IN(
    SELECT hospital_id
    FROM Doctors
    GROUP BY hospital_id
    HAVING COUNT (*) >= ALL (
        SELECT COUNT (*)
        FROM Doctors
        GROUP BY hospital_id
    )
);
```

(e)
```
SELECT H.h_name, COUNT (D.doc_id) AS total_doctors
FROM Hospital H
LEFT JOIN Doctors D
ON H.h_id = D.hospital_id
GROUP BY H.h_id, H.h_name;
```

SET

Answers to the question no: 3

(a) $\Pi_{doc_name,\ specialization}\ (\sigma_{specialization\ =\ 'Cardiology}(Doctors))$

(b) $\Pi_{doc_name,\ specialization}\ (\sigma_{city='Chittagong'}\ (Doctors \bowtie Doctor_Patient \bowtie Patient))$

(c) $\Pi_{doc_name}\ (\sigma_{city='Dhaka'}\ (Doctors \bowtie Doctor_Patient \bowtie Patient))$

(d) $R \leftarrow \gamma_{hospital_id;\ COUNT\ (doc_id) \rightarrow total}\ (Doctors)$

Now,

$\Pi_{h_name,\ total}\ (\sigma_{total>5}\ (Hospital \bowtie_{h_id\ =\ hospital_id}\ R))$

(e) $R \leftarrow \gamma_{hospital_id;\ COUNT\ (doc_id) \rightarrow total}\ (Doctors)$

$M \leftarrow \gamma_{MAX\ (total) \rightarrow max_total}\ (R)$

$H_{max} \leftarrow \sigma_{total\ =\ max_total}\ (R \times M)$

$\Pi_{doc_name}\ (Doctors \bowtie_{hospital_id\ =\ hospital_id}\ H_{max})$